Criteria	
Technical Requirements - MVP *If any item is marked "fail" in this section, this section will be considered a failure. The presenter must fulfill the requirements listed below.*	
1.1	A full-stack application (non-MERN).
1.2	The back-end and front-end applications implement JWT token-based authentication to sign up, sign in, and sign out users.
1.3	There are at least 2 types of users roles, e.g. admin, vendor, client, etc.
1.4	Authorization is implemented across the front-end and back-end. Guest users (those not signed in) should not be able to create, update, or delete data in the application or access functionality allowing those actions. This includes conditional rendering and access on both front-end and back-end.
1.5	Implemented at least 10 APIs for the back-end and the front-end has to use at least 10 APIs
1.6	The project has at least one data entity in addition to the User model. At least one entity must have a relationship with the User model.
1.7	The project has full CRUD functionality on both the back-end and front-end.
1.8	The front-end application does not hold any secret keys. Public APIs that require secret keys must be accessed from the back-end application.
1.9	(Optional) The project is deployed online so that the rest of the world can use it.
Code Convention Requirements *If more than two items are marked "fail" in this section, this section will be considered a failure. If two or fewer items are marked "fail", it is considered passed. Your app must fulfill the requirements listed below.*	
2.1	The files in the back-end and front-end applications are organized following the conventions demonstrated in lectures.
2.2	The code in the back-end and front-end applications adheres to the coding conventions demonstrated in lectures, like using plural names for lists and arrays.
2.3	The back-end and front-end applications do not contain dead code, commented-out sections, prints, or console logs.
2.4	The back-end application can be used without encountering errors in the terminal. The front-end application can be used without encountering errors in the browser's console.
2.5	The back-end application follows RESTful routing conventions for routes.
2.6	The back-end and front-end applications are coded using proper indentation.
Code Understanding Requirements If any item is marked "fail" in this section, this section will be considered a failure. The presenter must fulfill the requirements listed below.	
2.6	Adequately provide an understanding of project code, e.g. why they used a certain package, why they wrote the codes a certain way and not another.
2.7	Adequately explain project code when asked questions during the soft launch/check-in, pre and post presentation. Questions can include how the code works, why are you using a particular method or variable, what would happen if something is removed.
UI/UX Requirements *If more than two items are marked "fail" in this section, this section will be considered a failure. If two or fewer items are marked "fail", it is considered passed. Your app must fulfill the requirements listed below.*	
3.1	The application exhibits a visual theme, like a consistent color palette and cohesive layout across pages.
3.2	The application is easily navigable by a first-time user. For example, navigation should be done through links instead of having to type in a URL to navigate around the app.
3.3	The application utilizes CSS Flexbox and/or Grid and/or a CSS framework for page layout design.
3.4	Colors used in the app have appropriate contrast that meet the WCAG 2.0 level AA standard.
3.5	When editing an item, the form is pre-filled with that item's details.
3.6	Only the user who created a piece of data can see and interact with the UI for editing or deleting that data.
3.7	All images have alt text.
3.8	No text is placed on top of an image in a way that makes the text inaccessible.
3.9	All buttons are styled.
Git and GitHub Requirements *If any item is marked "fail" in this section, this section will be considered a failure. Your interactions with Git and GitHub must fulfill the below requirements.*	
4.1	You are shown as the only contributor to the project on GitHub.
4.2	The GitHub repositories used for the project must be named appropriately. For example, names like book-binder-back-end or wellness-tracker-front-end are appropriate, whereas ga-project or flask-react-project are not. The repo must be publicly accessible. Be sure to create the repos on your personal GitHub account and ensure it is public.
4.3	Your repos should have commits that date back to the very beginning of the project. If you start over with a new repo, do not delete the old one.
4.4	Commit messages should be descriptive of the work done in that commit.
README Requirements *If more than two items are marked "fail" in this section, this section will be considered a failure. If two or fewer items are marked "fail", it is considered passed. Your README must contain the items or sections below.*	
5.1	Screenshot/Logo: A screenshot of your app or a logo.
5.2	Your app's name: Include a description of your app and its functionality. Background info about the app and why you built it is a nice touch.
5.3	Getting started: This section should include a link to your deployed app, a link to your planning materials, and a link to the back-end repository.

5.4	Attributions: This section should include links to any external resources (such as libraries or assets) or AI/LLM questions you used to develop your application that require attribution. You can exclude this section if it does not apply to your application. Please refer to the AI Policy for attribution guidelines (APA Style)
5.5	Technologies used: List the principal technologies used by your application, for example: Python, JavaScript, and any major frameworks or libraries.
5.6	Next steps: Planned future enhancements (stretch goals).
5.7	The back-end README file contains the name of the project and a link pointing to the front-end repository on GitHub for more details on the project.
Presentation Requirements *If any item is marked "fail" in this section, this section will be considered a failure. Your presentation must fulfill the below requirements.*	
6.1	Present your project in front of the class on the scheduled presentation day.
6.2	The project you present is the project you were approved by your instructor to build.